

From Digital Systems to Microcontrollers

Introduction to ARM Cortex-M Microprocessors – Chapter 1

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

- 1** Why This Matters
- 2 Embedded Systems
- 3 Meet the Board
- 4 Inside the Computer
- 5 Tracing $z = x + y$
- 6 The Instruction Execution Cycle
- 7 From C to a Pin

Your gate circuits did only what their **wiring** told them. To change the behaviour, you had to rewire. A microcontroller is a digital circuit too, but a **program stored in memory** controls it. The same silicon can blink an LED, measure a temperature, or drive a motor. It depends only on the bits you load.

After this chapter, you can:

1. Define an embedded system and its design constraints.
2. Distinguish a **microprocessor** from a **microcontroller**.
3. Name the CPU functional units and the role of each.
4. Explain the address, data and control buses, and compute addressable memory.
5. Trace a short program through fetch-decode-execute.
6. Describe how one C statement changes a voltage on a pin.

This chapter introduces many new terms. Every section answers just one question. This question is the **thread** that connects everything that follows:

How does a program cause hardware to do something?

The one fact to carry out of this chapter

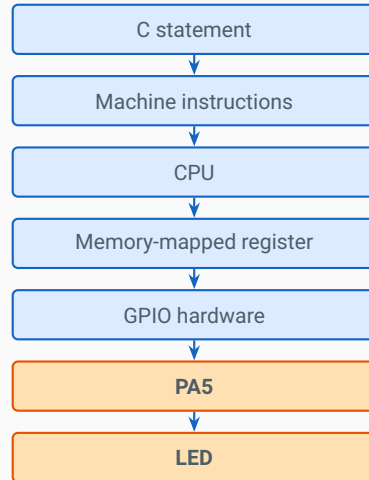
The CPU has no special instruction for “turn on an LED”.

It only has instructions that **read and write memory addresses**. Some of these addresses connect not to memory, but to hardware instead.

So turning on an LED means **writing a number to an address**.

Before we learn the vocabulary, let us first see where we are going.

By the end of this chapter, you will be able to explain **every arrow** in this chain. You do not need to understand it yet.



- 1 Why This Matters
- 2 **Embedded Systems**
- 3 Meet the Board
- 4 Inside the Computer
- 5 Tracing $z = x + y$
- 6 The Instruction Execution Cycle
- 7 From C to a Pin

General-purpose computer

- Runs many different applications
- The user can program it
- **Faster is always better**
- Sold on speed, compatibility, price

Embedded system

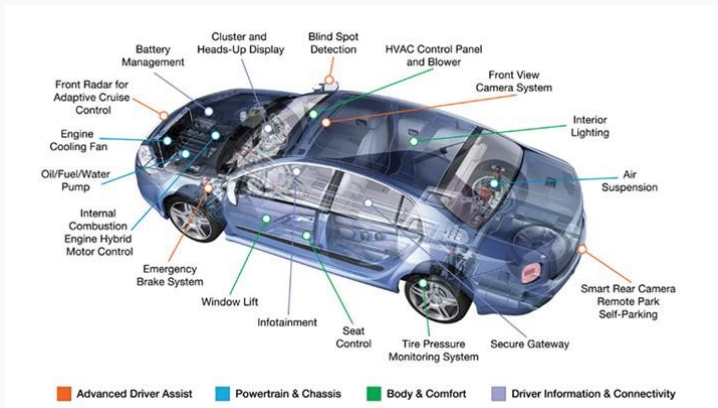
- Runs a few functions, fixed at design time
- The user cannot program it
- Must meet a **fixed deadline**; extra speed may not help
- Sold on power, cost, size, **predictability**

The idea that matters

A desktop that answers in 100 ms instead of 10 ms is just slow. But an airbag controller that fires in 100 ms instead of 10 ms has **failed**.

Embedded engineering cares about *predictable* time, not maximum speed.

Embedded Systems: An Example



Each label is at least one **ECU**: an **Electronic Control Unit**. It is a micro-controller, its firmware, and a network connection, all together.

A car today carries **50–150** of these. By **2030**, everyday cars are expected to carry **200–250**.

The driver never programs any of them directly.

Diagram credit: Embitel.

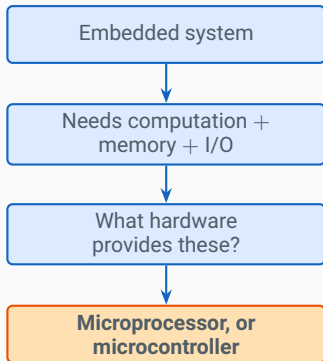
An ECU is classified by the function it controls: engine control module (ECM), body control module (BCM), electronic brake control module (EBCM), powertrain control module (PCM), transmission control module (TCM), suspension control module (SCM), door control unit (DCU), battery management system (BMS), and many more.

Constraint	Why it binds	What it forces
Size	Fits inside the product it controls	Package, pin count, board area
Weight and	Portable, airborne, wearable	Battery size, materials
Power	Battery life, thermal budget	Clock speed, sleep modes, duty cycle
Cost	Multiplied by production volume	Part choice, external components

Cost is multiplied, not added

Saving RM0.50 on one part looks trivial. But at 500,000 units, that saving becomes **RM250,000**. This is why embedded engineers argue about single resistors.

- 1 Why This Matters
- 2 Embedded Systems
- 3 **Meet the Board**
- 4 Inside the Computer
- 5 Tracing $z = x + y$
- 6 The Instruction Execution Cycle
- 7 From C to a Pin



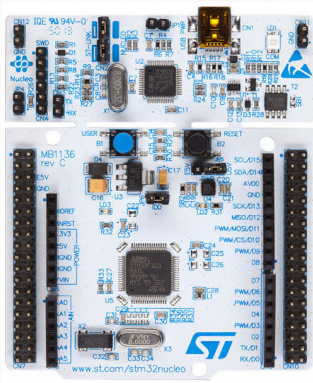
An embedded system is a *requirement*.
Something must satisfy that requirement.

The question this raises

What kind of computer belongs inside an embedded system?

We will answer this question using one real board, so every idea in this chapter has a physical home.

Meet the Board: STM32 Nucleo-F446RE



STM32 Nucleo-F446RE

- **ST-LINK** debugger — the upper third of the board, past the perforation. One USB cable handles power, programming, debugging, and a virtual COM port.
- **USER button** (B1) on **PC13**
- **User LED** (LD2) on **PA5**

The inner headers follow the Arduino shield layout. The outer **morpho** headers bring out every remaining pin.

Every blink program in this book drives PA5. Everything we discuss from here on will eventually explain this board.

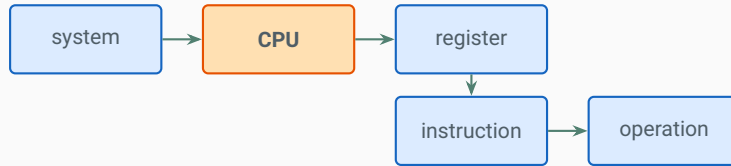
- 1 Why This Matters
- 2 Embedded Systems
- 3 Meet the Board
- 4 **Inside the Computer**
- 5 Tracing $z = x + y$
- 6 The Instruction Execution Cycle
- 7 From C to a Pin

Feature	Microprocessor (μ P)	Microcontroller (μ C)
Integration	Processor chip only	Processor, RAM, ROM, I/O on one die
Flexibility	Memory and peripherals scale	Resources fixed inside the chip
Complexity	Many components, complex PCB	Few components, simple board
Power, cost	Higher	Optimised for low power and unit cost
Performance	High clock, heavy computation	Lower clock, control and I/O

A microprocessor alone is useless

It has no RAM, no ROM, and no way to reach the outside world. All of this must be wired around it before it can do anything. A microcontroller puts exactly this all together on one chip. **The Nucleo's chip is a microcontroller:** everything on the previous slide sits inside one package.

The next few slides introduce many names at once: buses, registers, ALU, decoder. Do not think of this as a list to memorise. Instead, think of it as one picture, **zoomed in one level at a time**:

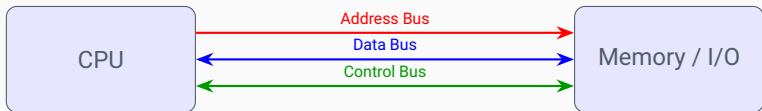


Level 1: the whole microcontroller

CPU RAM Flash GPIO UART ADC — all on one chip, connected by buses.

The buses let the CPU *reach* the rest of the chip. The registers are what the CPU is *made of*. Both sit one level deeper than “the microcontroller.”

The Three Buses



Bus	Direction	Carries
Address	Unidirectional, CPU → device	Which location. Width sets the addressable range
Data	Bidirectional	What value. Width sets how much moves per transfer
Control	Bidirectional	When and what kind: read/write, timing, interrupts

Address width sets the memory map

2^n addressable bytes for an n -bit address bus.

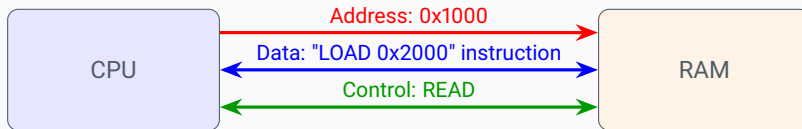
$2^{16} = 64 \text{ KB}$ · $2^{20} = 1 \text{ MB}$ · $2^{32} = 4 \text{ GB}$: the Cortex-M4

You already know this: an address is the input to a **decoder**. It works the same way as the one-of- N selection you built with a 3-to-8 decoder, just at a much larger scale.

Using the Buses: Fetching an Instruction

Suppose the CPU needs the instruction stored at 0x1000:

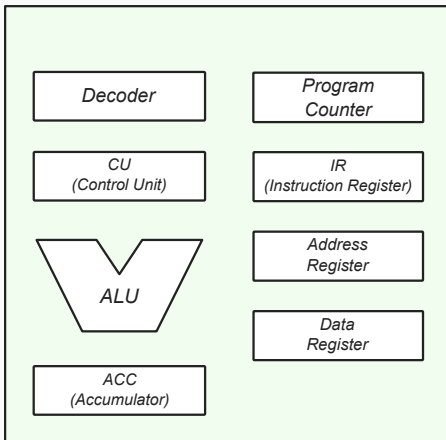
Bus	Carries, this step
Address bus	0x1000 – CPU asks for this location
Control bus	READ – CPU asserts a read
Memory	responds with LOAD 0x2000
Data bus	carries that instruction back to the CPU



The same pattern repeats for every data access

The address bus names the location. The control bus says read or write. The data bus carries the value. **Three buses, one transaction.** We will use this exact pattern again when we trace a program.

Processor



Memory

0x1000	LOAD 0x2000
0x1002	ADD 0x2002
0x1004	STORE 0x2004
0x1006	
.....	
.....	
0x2000	1
0x2002	2
0x2004	?

The registers in the centre connect to memory. The decoder, control unit and ALU on the left have **no direct memory access**. Everything they touch must first pass through a register.

Five names appear on the previous diagram. Not all of them need the same amount of attention yet:

Must understand now

- **PC** — address of the **next** instruction
- **IR** — the instruction just fetched
- **ACC** — operand and result of the ALU

Can recognise, needn't memorise yet

- **AR** — address of the data being accessed
- **DR** — value in transit to or from that address

Why “no direct memory access” matters

This is why **load/store** architectures exist. On the Cortex-M4, you cannot add two values that sit in memory directly. You must LDR them into registers, ADD them, then STR the result back. Chapters 5–7 explain this idea in more detail.

Chapter 2 replaces this generic register set with the Cortex-M4's actual sixteen registers. For now, just keep this simple picture in mind.

- 1 Why This Matters
- 2 Embedded Systems
- 3 Meet the Board
- 4 Inside the Computer
- 5 **Tracing $z = x + y$**
- 6 The Instruction Execution Cycle
- 7 From C to a Pin

One Line of C, Traced in Hardware

Consider the simplest possible program:

$$z = x + y;$$

Assume the compiler places x at $0x2000$, y at $0x2002$, and z at $0x2004$.

We will use this one line to discover what a CPU actually does

One line of C becomes three machine instructions. Every register and operation from the last few slides gets a **job** to do in making this happen.

Program memory

Address	Instruction
0x1000	LOAD 0x2000
0x1002	ADD 0x2002
0x1004	STORE 0x2004

Data memory

Address	Variable	Value
0x2000	x	1
0x2002	y	2
0x2004	z	?

Each new concept earns its place here by answering one question in this trace:

	Question	Answer
1	What is x?	value at 0x2000
2	How does the CPU obtain it?	LOAD
3	Where does it go?	ACC
4	How does addition happen?	the ALU
5	Where does the result go?	ACC
6	How does it become z?	STORE

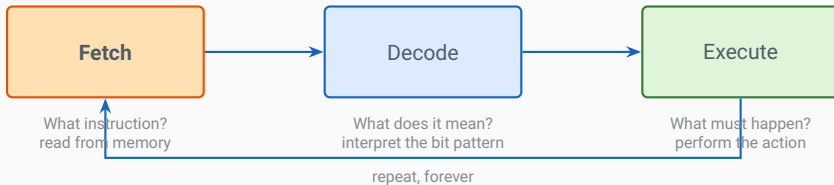
This is much easier to remember than a list

“Here are five registers, memorise what they do” gives these concepts no real job to do. **Every concept in the next three slides appears here because this trace needs it.**

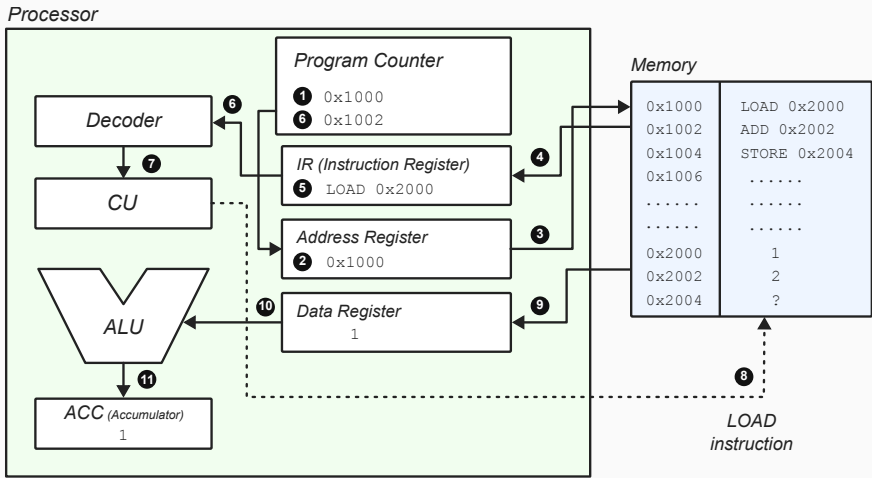
- 1 Why This Matters
- 2 Embedded Systems
- 3 Meet the Board
- 4 Inside the Computer
- 5 Tracing $z = x + y$
- 6 The Instruction Execution Cycle**
- 7 From C to a Pin

The CPU Has an Instruction. Now What?

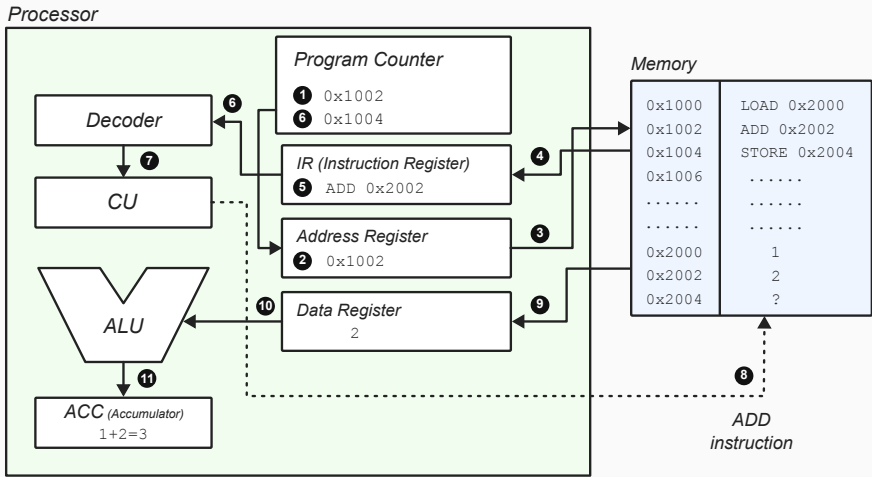
The CPU has fetched an instruction. What does it actually do with it?



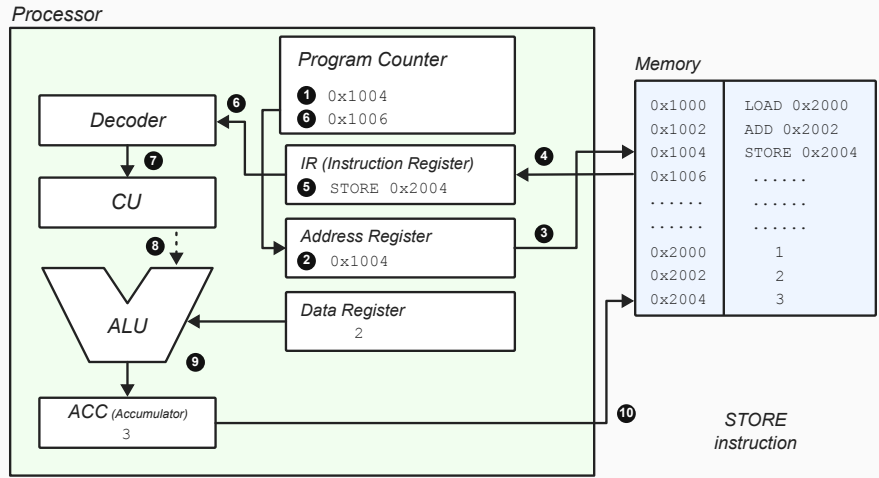
This is a finite state machine, driven by the clock. You built FSMs in digital electronics. **A CPU is the largest one you have met so far.** The loop never stops while the machine has power. This is why an embedded program's `main()` is an infinite loop *by design*.



The value 1 at address 0x2000 reaches the Accumulator. This answers questions 1–3 of the trace. Note step 6: the PC increases during decode, not after execute.



The value 2 is fetched, and the **ALU** adds it to the Accumulator. The Accumulator becomes 3. This answers questions 4 and 5.



The Accumulator's 3 is written out to 0x2004. This answers question 6. The program is now complete: three instructions, each made of several internal operations.

- 1 Why This Matters
- 2 Embedded Systems
- 3 Meet the Board
- 4 Inside the Computer
- 5 Tracing $z = x + y$
- 6 The Instruction Execution Cycle
- 7 From C to a Pin**

How does a program cause hardware to do something?

We just traced $z = x + y$; through LOAD, ADD, and STORE. These were all ordinary memory addresses, nothing more. Now let us return to the board: what happens when the address is not a memory cell, but a peripheral instead?

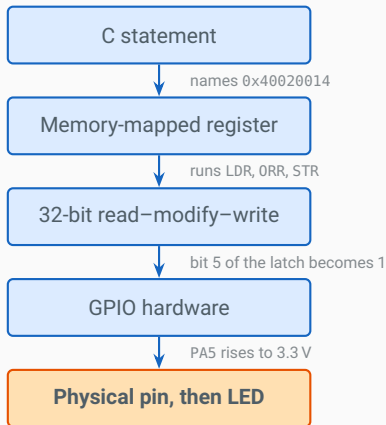
One Line of C, Back on the Board

Look at this line. It appears in almost every STM32 program ever written:

```
GPIOA->ODR |= (1U << 5);
```

You already know every symbol here: a pointer, a structure member, a compound bitwise-OR assignment, and a left shift.

What may not be obvious is that running this one statement **turns on an LED**: the LED on PA5, on the board you met at the start of this chapter.



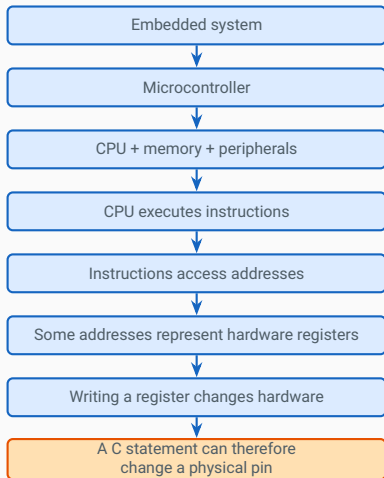
The CPU still does not know what an LED is

It only knows how to execute instructions and access addresses. Every step in that chain was ordinary: the same LDR/STR pattern as any other read-modify-write.

Turning on an LED is writing a number to an address.

This is the same sentence from “The One Question.” Now you fully understand it.

Summary: The Causal Chain



Terminology covered

- Embedded systems and SWaP-C
- Microprocessor vs microcontroller
- Buses
- CPU registers, ALU, control unit
- Fetch-decode-execute
- Memory-mapped I/O

Next: Chapter 2

Turns this generic CPU into the specific one on your bench.